Challenge Name: onnx-model-secrets

1. Recon

The handout is **not** the challenge, it is a 526-byte README with a single instruction and three hints:

> Download the model file and analyze it. Something sensitive was embedded during the model's **calibration process**.
>
> - The ONNX format is based on Protocol Buffers
> - Models can carry **more than just weights and graph structure**
> - Tools: `onnx` Python library, Netron

The instance serves a fake internal ML API:

```
B="https://<instance>.222.255.138.122.nip.io"
curl -sk "$B/api/info"
```


```
{"model_name":"EmotionNet-v2.1","version":"2.1.0","input_shape":[1,2304],
 "output_classes":
["angry","disgust","fear","happy","sad","surprise","neutral"],
 "framework":"PyTorch 2.1","dataset":"FER-2013","accuracy":0.72}
```


The actual artifact is at `/model.onnx` 2,430,204 bytes:

```
curl -sk "$B/model.onnx" -o model.onnx
```

`file` reports only `data`; ONNX has no magic bytes, it is a bare protobuf stream.

First move, `strings`**.** Before any tooling, this immediately pays off:

```
$ strings -n 6 model.onnx | grep -iE 'flag|secret|key|token|passw|calib'
calibration_method
calibration_tensor
calibration_indices
calibration_hash
```

Four keys matching the README's "calibration process" wording. No `flag` string, so the flag is not stored in the clear.

2. ONNX is protobuf, reading `metadata_props`

An ONNX file is a serialised `ModelProto`. The relevant top-level fields:

Field	#	Meaning
`ir_version`	1	IR version
`producer_name`	2	exporter
`graph`	7	`GraphProto` nodes + initializers (the weights)
`opset_import`	8	opset
`metadata_props`	**14**	**repeated `StringStringEntryProto` free-form key/value**

`metadata_props` is the "more than just weights and graph structure" from the hint: an arbitrary string map that survives export and that **no inference runtime ever reads**. It is the natural place to smuggle data.

`pip install onnx` pulls a large build, so rather than wait I decoded the protobuf wire format directly. It is ~20 lines, worth having in the toolbox, because it works on any protobuf artifact with no schema and no dependencies:

```
def varint(b, i):
    r = s = 0
    while True:
        c = b[i]; i += 1
        r |= (c & 0x7f) << s
        if not (c & 0x80): break
        s += 7
    return r, i

def fields(b, start=0, end=None):
    """Yield (field_number, wire_type, value) for one protobuf message."""
    i, end = start, len(b) if end is None else end
    while i < end:
        key, i = varint(b, i)
        fn, wt = key >> 3, key & 7
        if   wt == 0: v, i = varint(b, i); yield fn, wt, v          # varint
        elif wt == 2: ln, i = varint(b, i); yield fn, wt, b[i:i+ln]; i += ln
# bytes
        elif wt == 5: yield fn, wt, b[i:i+4]; i += 4                # fixed32
        elif wt == 1: yield fn, wt, b[i:i+8]; i += 8                # fixed64
        else: raise ValueError(f"wire {wt} at {i}")
```

Walking field 14 and splitting each entry into key (1) / value (2):

```
model_name          = EmotionNet-v2.1
author              = AI Research Lab
framework           = PyTorch 2.1
training_dataset    = FER-2013
accuracy            = 0.72
export_date         = 2025-11-03
calibration_method  = weight_indexed_xor
calibration_tensor  = fc2.weight
calibration_indices = 0,3,7,11,15,19,23,27,31,35,39,43,47,51,55,59
calibration_hash    =
d986fbb31248e5b9267f7ef34d60fcb4dbd8b7e05a47e4f02f7978a3517df8b7d9daaaec5a14e1
eb2233
```

The model ships its own decryption recipe. Three observations settle the shape of the puzzle before writing any solve code:

1. `calibration_hash` is **not a hash**, 84 hex chars = **42 bytes**, and a hash would not be 42 bytes. It is the ciphertext.
2. 42 bytes is exactly `len("flag{") + 36 (UUID) + len("}")` = 5 + 36 + 1. The plaintext is a `flag{<uuid4>}`.
3. `calibration_indices` has **16 entries** → a 16-byte key, repeating over 42 bytes.

`weight_indexed_xor` names the scheme: index into a weight tensor, use it as XOR key material.

3. Extracting `fc2.weight`

Weights live in `GraphProto.initializer` (field 5), each a `TensorProto`: `dims`=1, `data_type`=2, `name`=8, `raw_data`=9. `data_type == 1` is FLOAT32, and `raw_data` is the little-endian float array.

```
graph name: EmotionNet
  'fc1.weight'    dims=[128, 2304] raw=1179648
  'fc1.bias'      dims=[128]       raw=512
  'fc2.weight'    dims=[64, 128]   raw=32768     <-- calibration_tensor
  'fc2.bias'      dims=[64]        raw=256
  'fc3.weight'    dims=[7, 64]     raw=1792
  'fc3.bias'      dims=[7]         raw=28
  'fc1.weight_t'  dims=[2304, 128] raw=1179648
  'fc2.weight_t'  dims=[128, 64]   raw=32768     <-- decoy
  'fc3.weight_t'  dims=[64, 7]     raw=1792
```

`fc2.weight` is 8192 float32 values; the indices are flat offsets into it.

⚠ The transpose trap

Every weight appears **twice**, once as `X` and once as `X_t`. These are genuine transposes (verified: `fc2.weight_t == fc2.weight.T`, exactly). Same 8192 values, different order, so picking the wrong one silently produces garbage.

What makes this a nasty trap rather than an obvious one: **element `[0,0]` is shared between a matrix and its transpose**, so index 0 gives the same key byte either way, and the wrong tensor still decrypts the first character to a correct `f`:

```
fc2.weight    -> b'flag{96d117a1-60d2-4367-8711-023f0083e265}'
fc2.weight_t  -> b'f\xe6\xda\xfd,P\xe9Y\xf9&:g\xe9\x9a\x11\nd\xb8...'
                  ^ correct first byte, everything after is noise
```

Seeing `f` followed by junk reads like "nearly right, wrong offset" and invites fiddling with the index list. It isn't, it is the wrong tensor. `calibration_tensor` says `fc2.weight`, and it means it.

4. The key derivation the actual puzzle

The metadata never says *how* a float becomes a key byte. The 16 selected values:

```
w[ 0] = -0.020146249    raw(LE) = bf 09 a5 bc
w[ 3] =  0.018921811    raw(LE) = ea 01 9b 3c
w[ 7] =  0.055274583    raw(LE) = 9a 67 62 3d
w[11] =  0.034938650    raw(LE) = d4 1b 0f 3d
...
```

Rather than guessing derivations (`int(v*255)`, `abs(v)*1000 & 0xff`, …), use the **known plaintext**. The flag starts `flag{`, so the first key bytes are forced:

```
key[0] = ct[0] ^ 'f' = 0xd9 ^ 0x66 = 0xbf
key[1] = ct[1] ^ 'l' = 0x86 ^ 0x6c = 0xea
key[2] = ct[2] ^ 'a' = 0xfb ^ 0x61 = 0x9a
```

Compare against the raw float bytes above: `bf`, `ea`, `9a` the **first byte of each float32's little-endian representation**. Solved by inspection, no brute force.

That byte is the **low 8 bits of the 23-bit mantissa**:

```
-0.020146249  =  0xBC A5 09 BF   (big-endian bit view)

   S  EEEEEEEE  MMMMMMMMMMMMMMM MMMMMMMM
   1  01111001  010010100001001 10111111
                                └──────┘
                                mantissa[7:0] = 0xBF
                    = little-endian byte 0 = the key byte
```

So the keystream is the least-significant mantissa byte of 16 chosen weights, the noisiest, least meaningful bits in the entire model.

5. Solve

```python
import numpy as np, struct

w   = fc2_weight                        # 8192 float32, from GraphProto.initializer
idx = [0,3,7,11,15,19,23,27,31,35,39,43,47,51,55,59]
ct  = bytes.fromhex("d986fbb31248e5b9267f7ef34d60fcb4"
                    "dbd8b7e05a47e4f02f7978a3517df8b7"
                    "d9daaaec5a14e1eb2233")

# key byte = low mantissa byte = byte 0 of the little-endian float32
key = bytes(struct.pack('<f', float(w[i]))[0] for i in idx)
pt  = bytes(c ^ key[i % len(key)] for i, c in enumerate(ct))
print(key.hex(), pt.decode())
```